İSTANBUL TOPKAPI ÜNİVERSİTESİ

MAY 2026

Multi-User JavaFX Weather Forecasting and Identity Persistence Application

Engineering Faculty / Computer Engineering

Programming II

	Student Name and Surname	Student ID	Signature
1	Saliha KAPO	25040102005	

1. INTRODUCTION

This project is a desktop-based Weather Application developed using Java 11 and the JavaFX GUI framework. The application allows multiple users to safely create individual accounts, securely log into the system, dynamically query real-time global weather data via the OpenWeatherMap API, and monitor their personalized search histories.

The application retrieves real-time weather data from the OpenWeatherMap API and displays information such as temperature, humidity, weather description, and weather icons. Additionally, a dynamic 5-day weather forecasting dashboard is integrated below the real-time weather status to provide extended prognostic data.

The core architectural milestone achieved in this development lifecycle is the successful bridge between local secure data persistence frameworks and external RESTful JSON enterprise web streams. To comply with the Single Responsibility Principle (SRP) and ensure maximum security, the user authentication infrastructure is designed with fully decoupled controllers for Login and Sign-Up operations. Furthermore, the identity layer incorporates an enterprise-grade cryptographic pipeline inside PasswordUtil.java, combining unique SecureRandom salting with iterative PBKDF2WithHmacSHA256 hashing to deliver an error-immune, robust security system.

2. PROJECT FEATURES

Main features of the application:

- **Decoupled User Registration System:** An autonomous account creation pipeline managed by independent interface boundaries to handle secure onboarding.
- **User Login/Logout System:** A session management mechanism focused strictly on credential validation and runtime state security.
- **Weather Data Retrieval using API:** Establishing secure network connections targeting remote web service endpoints to ingest real-time regional climate statistics.
- **Real-Time Weather Information Display:** Active mapping of real-time environmental metrics directly into readable UI fields.
- **Weather Icons Based on Weather Conditions:** Graphic state matching that visualizes current atmospheric data using conditional weather icons.
- **Search History Recording:** Append-only physical data logging tracking chronologically sequential query events.
- **User-Specific History Display:** Enforcing strict profile isolation boundaries during runtime execution to restrict users exclusively to their own historical queries.
- **Clear History Functionality:** Multi-user data safety compliance enabling individual users to flush their respective tracking logs without corrupting the historical files of other profiles.
- **Invalid Login and Invalid City Handling:** Defensive input enclosures deploying contextual warning alerts to protect execution threads from unmapped city values and data-entry errors.
- **JavaFX Graphical User Interface:** A fully styled, responsive desktop layout leveraging integrated containers, FXML structures, and component bounds for optimal user engagement.

- **Advanced Binary Data Persistence (users.dat):** Utilizing native Java Object Serialization to securely compress, isolate, and maintain local identity models against manual external configuration errors.
- **Dynamic 5-Day Weather Forecasting Layer:** An advanced chronological forecasting matrix rendered horizontally below the real-time weather components to project multi-day climatological data points.

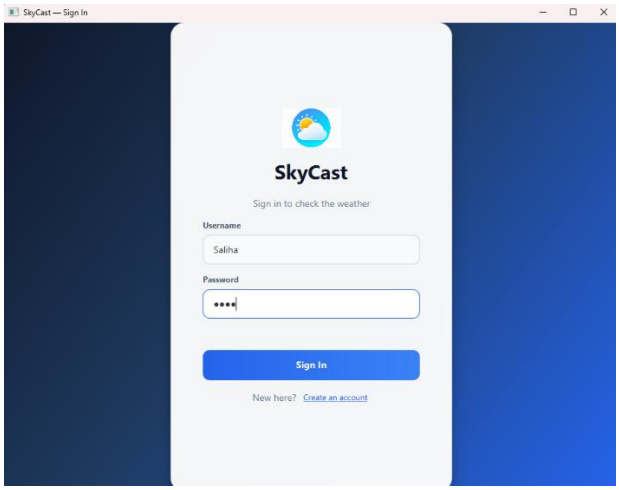

Figure-1: Login screen of the Weather Application. This screen allows existing users to authenticate using their username and password. The interface was designed using JavaFX components and includes input validation and user-friendly styling.

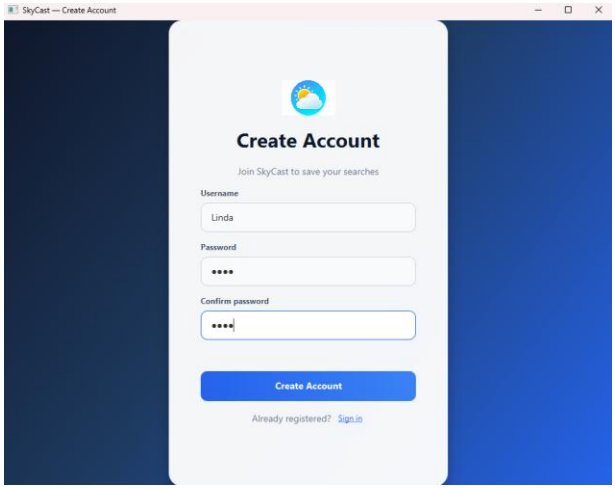

Figure-2: User registration process. New users can create accounts using the registration system. User credentials are securely marshaled and stored locally within the users.dat data file via Object Serialization after validation checks are completed successfully.

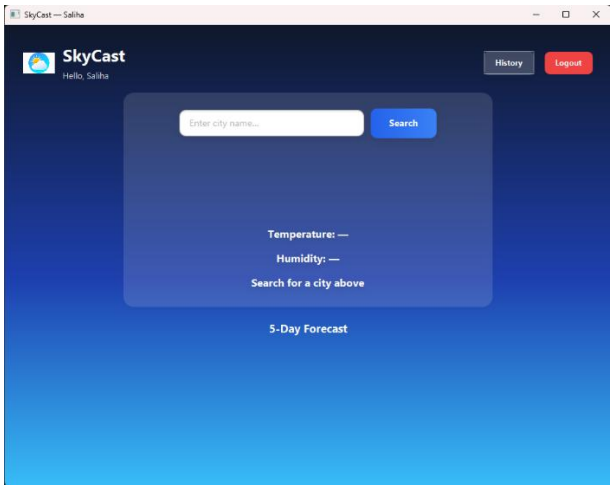

Figure-3: Weather information retrieval screen. Users can enter a city name and retrieve live weather information through the OpenWeatherMap API. The application displays temperature, humidity, weather description, and weather icons dynamically.

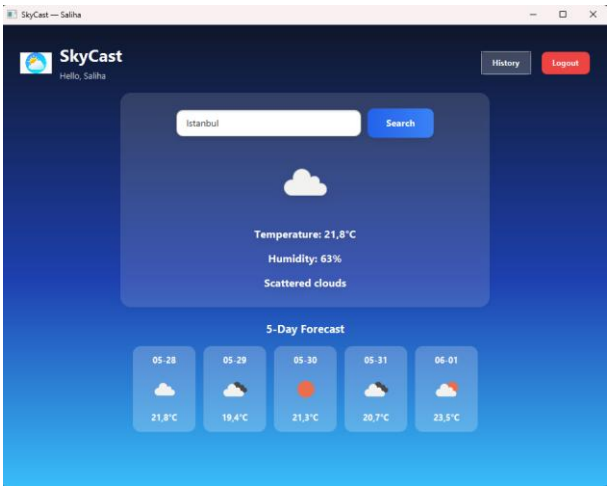

Figure-4: Example weather search result. The application successfully retrieves and displays real-time weather data obtained from the external API service. Weather conditions are visually supported using dynamic weather icons over the sequential 5-day forecasting dashboard layer.

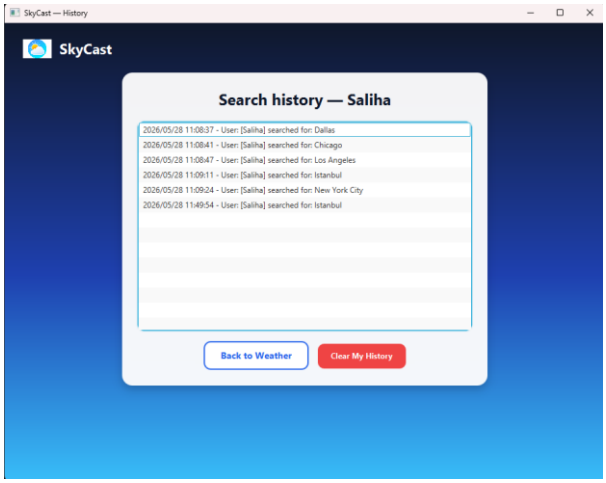

Figure-5: User-specific search history screen. The application stores weather search records in a local text file and displays only the active user's history using a JavaFX ListView component.

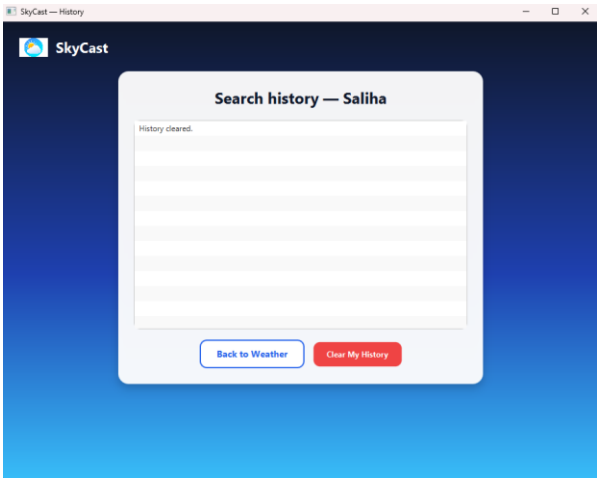

Figure-6: Clear history functionality. Users can remove their personal search history without affecting the records of other users. The system updates both the interface and the local history file dynamically.

3. TECHNOLOGIES USED

Technologies Used:

- Programming Language: Java 11
- GUI Framework: JavaFX (FXML & Layout Managers)
- Build Automation Tool: Maven
- External Service Endpoint: OpenWeatherMap API
- Data Parsing Library: JSON Java Library (org.json)
- Identity Cryptography Engine: Java Cryptography Architecture (PBKDF2WithHmacSHA256)
- Persistence Layers: Java Object Serialization API (users.dat) and character-buffered TXT file streaming (history.txt).
- Integrated Development Environment (IDE): IntelliJ IDEA

4. SYSTEM DESIGN & ARCHITECTURE

The system closely complies with the Controller-based architecture using structural JavaFX FXML representations and decoupled Java controller packages.

Main Components & Class Responsibilities:

• WeatherApplication.java: The core application bootstrap class initializing the fundamental stage, layout boundaries, and global window views.

• LoginController.java: Explicitly isolated to govern existing user session verification requests, checking credentials against the localized binary data repository.

• RegisterController.java: A dedicated controller spawned to manage user account creation pipelines independently. This satisfies the Single Responsibility Principle (SRP) by decoupling registration constraints from the login view state.

• WeatherController.java: Dispatches synchronous HTTP web requests to remote REST api channels, controls JSON element extractions, and maps real-time climate data into UI components.

• HistoryController.java: Controls the search tracking stream, dynamically parsing the global ledger to display user-restricted histories inside active JavaFX components.

• PasswordUtil.java: An isolated cryptographic utility executing key derivation functions, salting generations, and timing-attack immune password match evaluations.

5. AUTHENTICATION SYSTEM

The application features a secure authentication infrastructure split into two standalone layers to comply with the Single Responsibility Principle (SRP): an entry module managed by LoginController.java and an account creation engine driven by RegisterController.java.

Instead of using insecure plain-text character files, user profiles are encapsulated and stored in a local binary repository named users.dat using Java Object Serialization.

Input Validation and Security Safeguards

Before any user data is written to the binary store or evaluated for login, the system executes a sequence-based validation pipeline directly within the user interface:

- **Empty Field Prevention:** The interface intercepts submission requests dynamically. If any input fields are left blank, the authentication transaction is aborted instantly to prevent corrupted data structures.
- **Case-Insensitive Duplicate Control:** During registration, the system scans the existing collection framework inside users.dat. Username matching is strictly case-insensitive. If a duplicate account identifier is detected, the process terminates with a warning alert to prevent profile overwriting.
- **Password Length Constraint:** To maintain structural complexity, the registration tier automatically enforces a minimum password length rule. Inputs failing to meet this index are rejected before reaching the cryptographic engine.

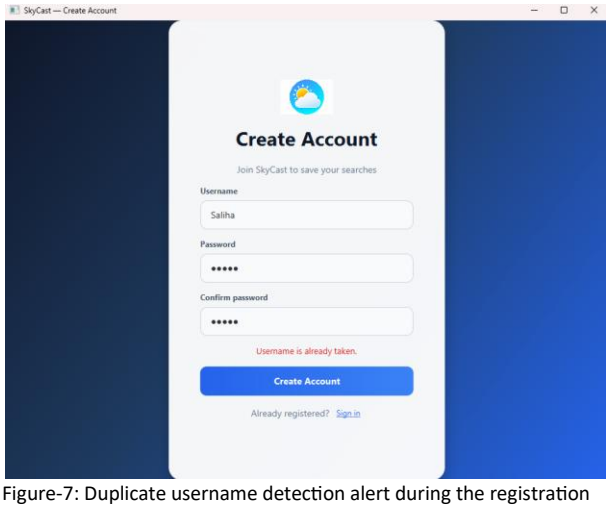

Figure-7: Duplicate username detection alert during the registration process. The system blocks account creation and displays a warning message if the identifier already exists in users.dat.

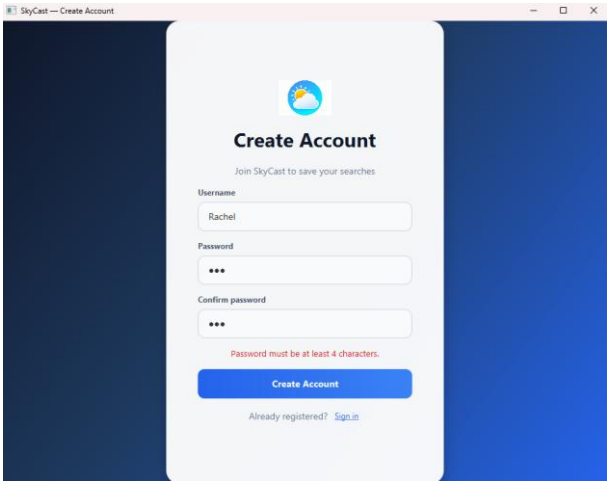

Figure-8: Minimum password length validation boundary. The interface rejects short passwords and prompts the user with an appropriate warning message on the status label.

6. WEATHER API INTEGRATION

The application establishes synchronous connection pipelines with the OpenWeatherMap external REST framework to fetch real-time atmospheric data. By leveraging native networking protocols and structured data parsers, the integration layer transforms raw web streams into dynamic dashboard metrics.

Technical Workflow and Ingestion Pipeline

The web communication layer processes external queries through a multi-stage validation and network loop:

- **Network Stream Initialization:** When a valid city name is passed by the controller, the system configures a secure connection layout using an active HttpURLConnection node to issue remote HTTP GET requests.
- **Data Parsing and Extraction:** The incoming string character sequence is processed into an object structure utilizing the org.json library. The application traverses the payload to isolate specific key-value targets:
    - Temperature: Extracted from the primary thermal data blocks.
    - Humidity: Retrieved to show the local environmental moisture ratio.
    - Weather Description: Parsed from the weather condition array to display text status updates.
    - Weather Icon Code: Collected to cross-reference localized graphical resources.
- **Dynamic Content Mapping:** The application dynamically updates the user interface properties. The climate icon code is bound to a JavaFX ImageView component, allowing the system to update condition graphics instantly without lagging the interface.
- **Chronological Forecast Ingestion:** Beyond real-time data tracking, the controller reads the extended predictive array indexes. It slices nested array layers into 5 distinct chronological segments, generating an automated, horizontal multi-day forecasting sequence embedded inside a JavaFX ScrollPane container

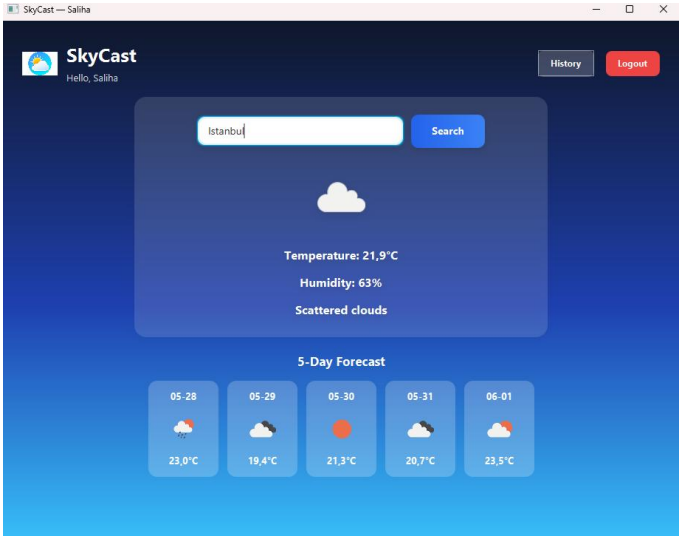

Figure-9: Example of weather data retrieval using the OpenWeatherMap API. The application successfully retrieves real-time weather information and displays temperature, humidity, descriptive state updates, and synchronized condition icons dynamically over the responsive 5-day meteorological forecasting carousel.

7. ADVANCED DATA PERSISTENCE & FILE ARCHITECTURE

The application implements a robust, hybrid data persistence layer carefully divided into customized binary object streams for secure authentication and text-based structured logging for multi-user search tracking.

A. Advanced Object Serialization Repository (users.dat)

To eliminate the vulnerabilities associated with storing user credentials inside unencrypted, line-by-line structured plain text files, the identity layer natively utilizes **Java Object Serialization**. A dedicated, encapsulated data model named User.java handles this mechanism, implementing the Serializable interface with a fixed tracking token (serialVersionUID = 1L).

- Registration Pipeline (ObjectOutputStream): During the account creation process, newly instantiated user entities—containing unique user identifiers and cryptographically secure hashes—are appended into a collection framework (List<User>). This aggregated object graph is deeply marshaled into a secure binary data container named users.dat utilizing an ObjectOutputStream node.
- Authentication Pipeline (ObjectInputStream): When a session verification transaction is initiated during login, the database tier decompresses the persistent binary stream via an ObjectInputStream to reconstruct the collection back into active memory instances (Deserialization).
- Data Integrity and Tamper Protection: This advanced design pattern guarantees comprehensive data integrity. If an external text editor attempts to open or maliciously modify the raw byte characters inside users.dat, the platform environment will raise a StreamCorruptedException at runtime, instantly breaking the unauthorized transaction and isolating the database layer from tampering.

B. Text-Based Structured Search Logging (history.txt)

While user identities are fully secured via encapsulated binary serialization maps, the application leverages local text file processing for user-specific activity tracking to maintain clear runtime logging metrics .

- File Used**:** history.txt
- Operational Purpose: It acts as an append-only shared physical ledger that persistently logs search operations for all users combined with precise date-time stamps. The application implements user-specific stream filtering during runtime using the active session name so that each user is exclusively restricted to viewing their personal search activities inside the JavaFX ListView component.
- Structured Format Example**:** 2026/05/28 13:10:19 - User: [Saliha] searched for: Ankara

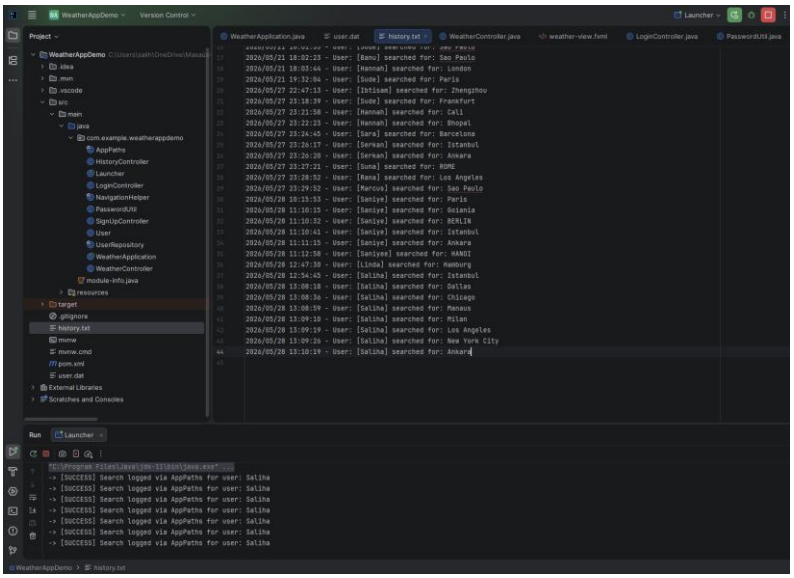

Figure-10: Example of file-based history storage. The application uses local text file processing to persist user search history data. Each record contains the username, searched city, and timestamp information, enabling user-specific history management.

8. HISTORY SYSTEM

The application manages user-specific query metrics through an isolated filtering layer, ensuring that history logs remain secure and contextually separated between different user profiles. Even though search records are collected within a shared storage infrastructure, individual session states protect user privacy at runtime.

Multi-User Isolation and Data Flushing Mechanics

The tracking framework operates using two main interface and file-level constraints:

- **Session-Based Stream Filtering:** When an authenticated user opens the history panel, the system does not load the entire raw text ledger. Instead, it intercepts the data stream and filters entries based on the active username token. Only matching records are bound and rendered inside the JavaFX ListView component.
- **Selective Deletion Operations:** The clearing feature allows an active user to wipe their own history footprint without altering or corrupting the search records belonging to other registered accounts. The background engine selectively removes the user's entries from the physical file and updates the visual interface state simultaneously.

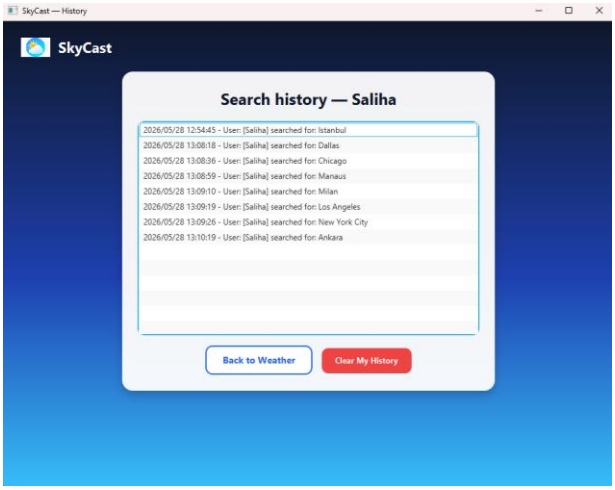

Figure-11: User-specific search history display. After successful authentication, the application displays only the active user's search history records by filtering data according to the logged-in username sub-string.

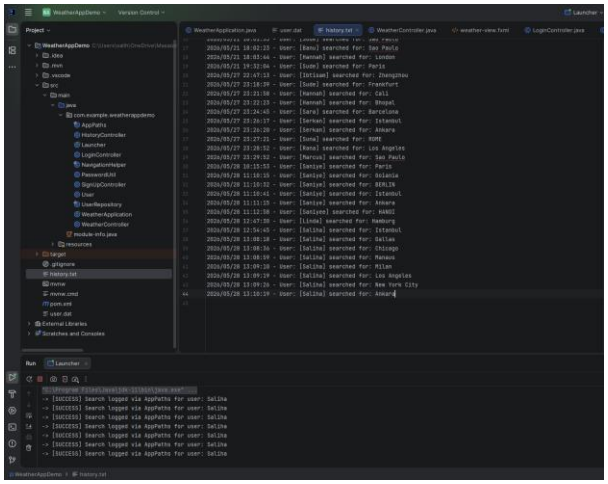

Figure-12: Complete history records stored in history.txt. The physical ledger persistently logs all search rows combined with absolute timestamps and username tags for background tracking purposes.

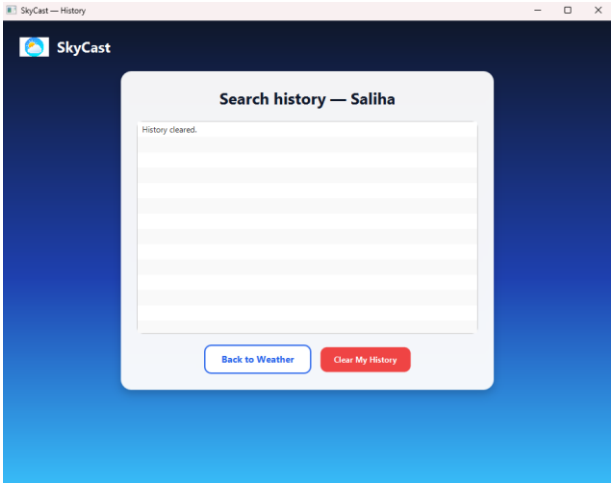

Figure-13: Clear history operation triggered by the active user. The interface provides a dedicated action button allowing individual profiles to request a complete reset of their personal query logs.

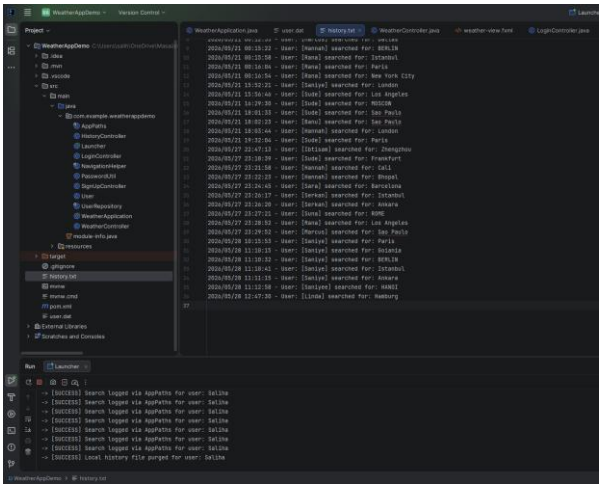

Figure-14: Updated state of the history.txt ledger after selective history deletion. The application dynamically removes the active user's lines while preserving the sequential log data of all other accounts.

9. ERROR HANDLING

The application implements comprehensive error handling and input validation mechanisms to maintain thread stability, prevent unexpected system crashes, and ensure a robust user experience across all modules. By intercepting bad inputs and network exceptions at the interface level, the system protects both local and remote data streams.

Key Validation and Exception Enclosures

The exception handling engine operates through several critical checkpoints:

- **Empty Input Enforcements:** The interface layer checks text properties before processing queries or authentication requests. Blank fields trigger immediate visual validation alerts, preventing unnecessary network transmissions or empty storage creation.

- **Geographical Response Resolution (Invalid City):** If the OpenWeatherMap endpoint returns an error code indicating a non-existent or misspelled city, the application catches the bad code gracefully. It dynamically updates the status label with a contextual warning rather than allowing the layout properties to fail or display corrupted data.
- **Authentication Failure Safeguards:** The session management tier cross-checks credential models using precise criteria. Incorrect passwords or missing usernames immediately trigger status warnings, successfully blocking unauthorized application navigation.
- **Identity Duplication Controls:** The system scans the database arrays to enforce unique profile identifiers. Duplicate entries are intercepted at the boundary level to keep data accounts completely separate.
- **Structural Length Validation:** Password inputs are evaluated based on precise index boundaries. Short inputs are instantly rejected at the interface layer before reaching the cryptographic modules.
- **Password Match Verification (Newly Added):** During the account creation workflow within the standalone registration view, the interface validates whether the string properties in the "Password" and "Confirm Password" fields are identical. If a mismatch is detected, the pipeline is safely aborted before triggering any cryptographic computations, guarding the data layer against structural entry defects.
- **Network Stream Exception Handling:** All web communications are wrapped inside secure try-catch structures. API timeouts, missing internet handshakes, or server drops are handled safely, displaying informative warning states without disrupting application stability.

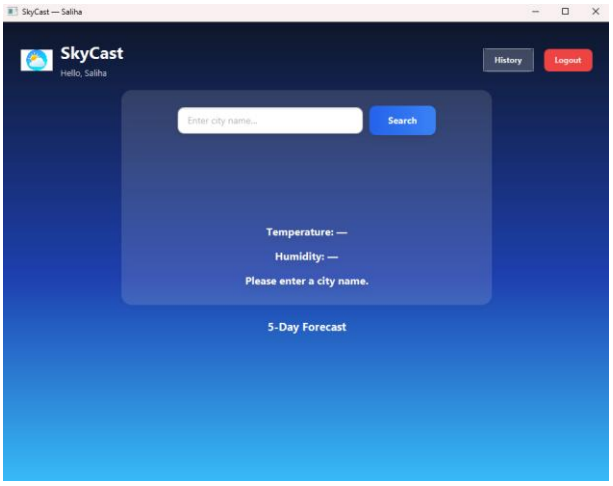

Figure-15: Empty city validation state. The application verifies whether the city input field is empty before making an API call, blocking the pipeline to prevent invalid communication overhead.

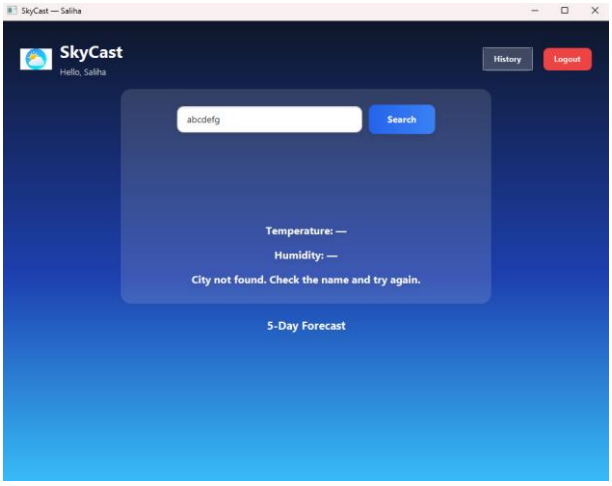

Figure-16: Invalid city detection warning. If the API returns a failure code, a clear warning is rendered dynamically over the 5-day forecasting layout instead of crashing the execution thread.

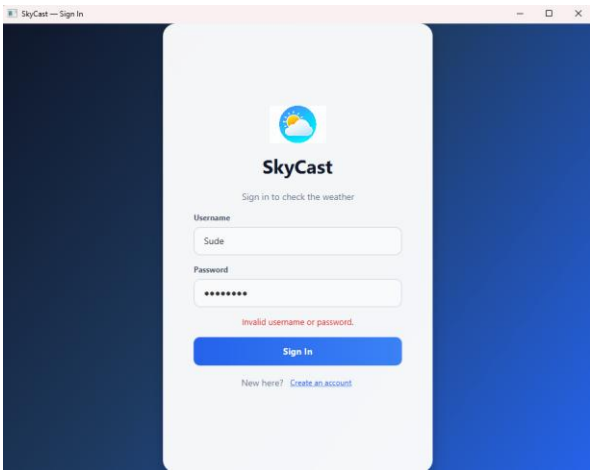

Figure-17: Invalid login detection. The login system checks entered usernames and passwords against the locally stored user records within the serialized users.dat file.If incorrect credentials are entered, the application denies access and displays an appropriate warning message.

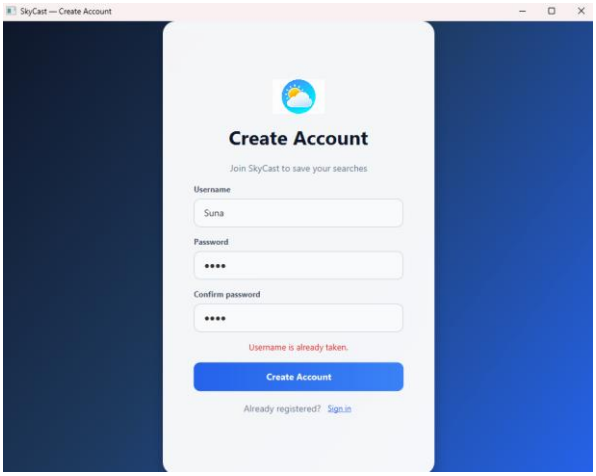

Figure-18: Duplicate username detection. During registration, the application checks whether the selected username already exists in the system. Duplicate usernames are prevented in order to maintain unique user accounts and avoid authentication conflicts.

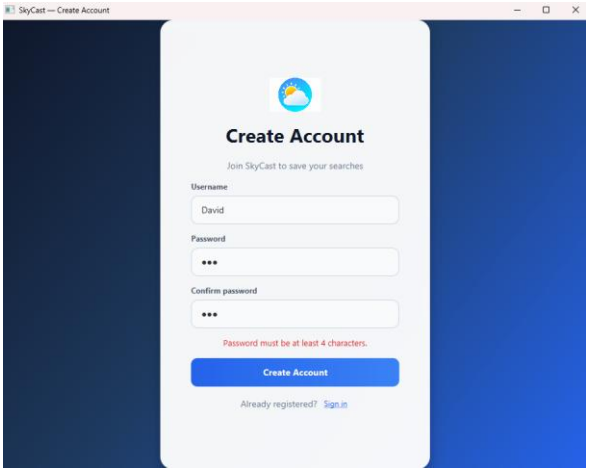

Figure-19: Minimum password length validation. The application validates password length during user registration. Passwords shorter than the required minimum length are rejected to improve account security and data consistency.

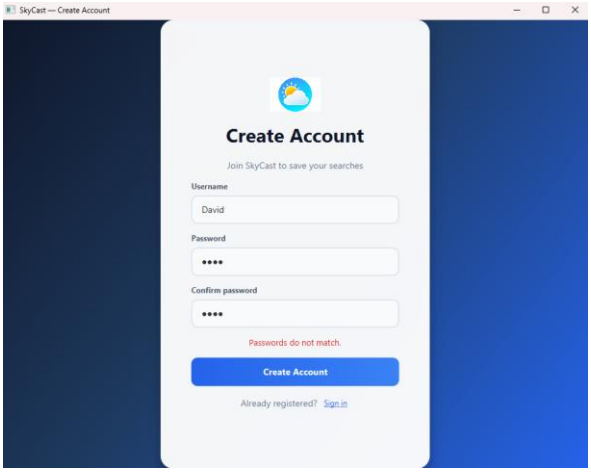

Figure-20: Password match confirmation validation layout on the standalone registration interface. The system actively evaluates the sequence of characters between the input field and the confirmation field. If the tokens do not match, the submission thread is restricted, and a prominent red visual warning ("Passwords do not match.") is rendered dynamically below the entry fields.

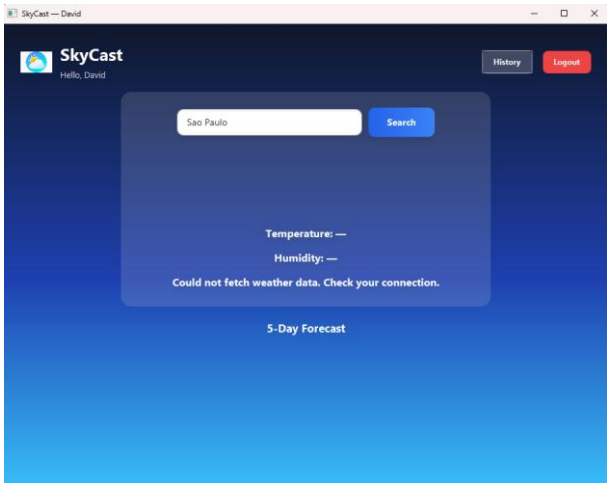

Figure-21: API connection exception handling visualization. If an internet or endpoint failure occurs, the application catches the event and communicates the network status safely to the user.

10. CRYPTOGRAPHIC IDENTITY PROTECTION & PASSWORD SECURITY

The application mitigates security vulnerabilities associated with credential management by replacing high-risk plain-text logging files with cryptographic salted hashing layers and encapsulated binary models. Raw user passwords are completely isolated from storage components and never persist dynamically in a clear, human-readable layout.

A. Modular Cryptographic Hashing Engine (PasswordUtil.java)

To comply with standard enterprise design principles and honor the Single Responsibility Principle (SRP), all cryptographic mechanics are completely isolated inside a dedicated utility package under the PasswordUtil.java modüle.

- **Secure Salting Mechanism:** During user account creation, the system triggers a SecureRandom cryptographic pseudo-random number generator to generate a unique 16-byte cryptographically secure "salt" array for every individual profile. This ensures that identical password entries produce entirely distinct cryptographic signatures across different accounts.

- **Iterative PBKDF2 Processing:** The explicit raw password characters combined with the unique salt are passed through the PBKDF2WithHmacSHA256 algorithm over **120,000 computation loops**. This multi-pass derivative mechanism guarantees mathematical protection against modern hardware-accelerated brute-force and pre-computed rainbow-table vector attacks.

- **Synchronized Verification Alignment:** During subsequent login workflows, the verification pipeline securely unpacks the exact 16-byte salt bytes saved within the serialized database stream. The candidate password string is dynamically routed back through the cryptographic core using these identical historical parameters, guaranteeing perfect verification consistency and an error-immune evaluation pipeline.

- **Timing-Attack Resistant Verification:** The evaluation is handled via MessageDigest.isEqual to enforce a time-constant character comparison. This systematically prevents information leakage

via precise execution timing analysis, blocking side-channel exploits at the system boundary.

B. Storage Encapsulation Layout via Java Object Serialization

Once the secure tokens are generated by the cryptographic layer, user attributes are packaged within an instance of the encapsulated User entity model. Rather than appending these items directly onto standard text logs, the identity subsystem writes the complete user matrix graph into a secure, protected binary system data store named users.dat.

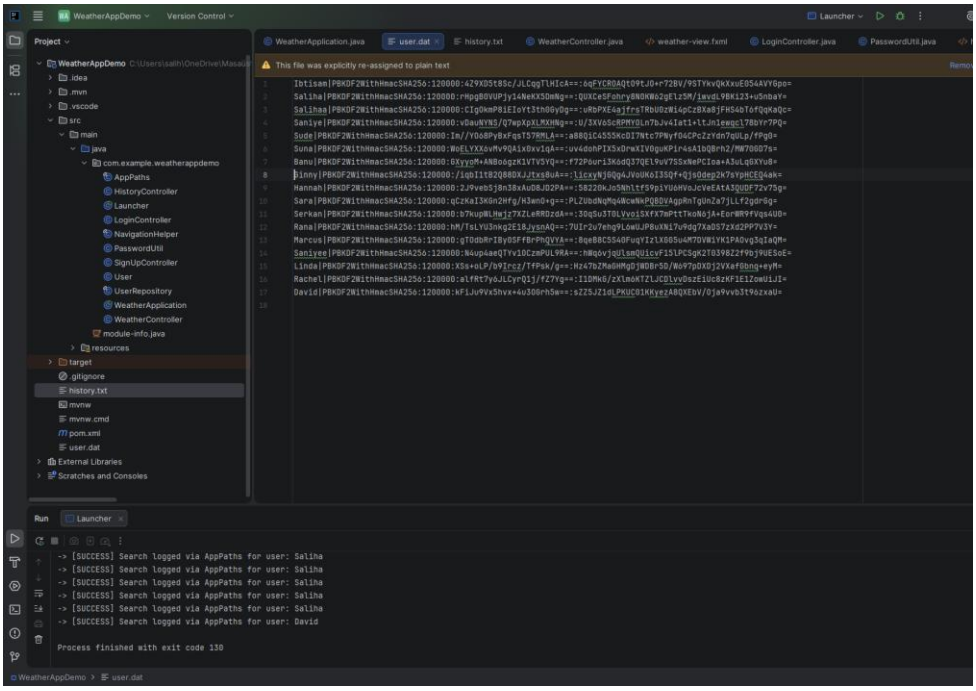

Figure-22: The serialized binary footprint inside users.dat. As visualized inside the IDE terminal, the system completely avoids generating any human-readable string patterns or plain-text credentials. Even if an adversary intercepts or attempts to open the underlying file structure, user credentials remain deeply protected under multi-layer cryptographic defenses (PBKDF2) and Java object encapsulation boundaries.

11. USER INTERFACE

The graphical user interface was developed using JavaFX FXML files to provide a modern, visually engaging, and user-friendly desktop experience. To maintain a clean architectural structure, separate and independent views were fully designed for the login, **account registration (Sign-Up)**, weather forecasting, and search history operations.

The interface leverages advanced JavaFX layouts and styling components, including:

- **Modern UI Styling:** Rich gradient backgrounds, synchronized color palettes, and custom-styled button layouts that ensure a cohesive visual language across all scenes.
- **Responsive Visual Components:** Dynamic weather condition icons mapped directly via ImageView fields, alongside a clean, custom horizontally-aligned layout structure configured to render the sequential 5-day predictive forecasting cards synchronously.

- **Smooth Scene Transitions:** Orchestrated navigation workflows implemented within the controller layers to allow seamless state switching between authentication views and the main weather dashboard.

12. APPLICATION WORKFLOW

The general operational workflow of the application executes through the following sequential stages:

1. **Application Initialization:** The user opens the desktop application, and the bootstrap layer displays the primary secure Login interface.
2. **User Onboarding / Account Creation:** If the user does not possess valid credentials, they navigate to the standalone registration view managed independently by the RegisterController to safely create an account.
3. **Identity Authentication:** The existing user enters their account identifiers on the separate Login screen to verify their credentials against the serialized users.dat binary database.
4. **Dashboard Redirection:** After successful cryptographic authentication, the application performs a smooth scene switch and displays the main personalized Weather Forecast dashboard.
5. **City Query Submission:** The authenticated user enters a target city name into the interface input field and triggers the query.
6. **Remote REST Communication:** The background networking layer dispatches a synchronous request to the external OpenWeatherMap API using an active connection stream.
7. **JSON Payload Breakdown:** The structural climate data response is successfully received and parsed from its raw JSON formatting layers.
8. **Synchronous UI Rendering:** The parsed metrics—consisting of the current temperature, humidity levels, condition description, and dynamic weather icon—along with the subsequent **5-day forecast cards** are rendered horizontally on the screen layout.
9. **Persistent Search Tracking:** The complete search operation is logged automatically into the shared history.txt ledger along with unique username tags and precise timestamps.
10. **Filtered Activity Auditing:** The user can navigate to the history panel to view or selectively clear their personal search history logs through runtime session filters.
11. **Session Termination:** The user may safely trigger a logout action, destroying the active session state and returning securely to the initial Login window view.

13. CONCLUSION

Throughout the development process, critical software engineering concepts were successfully implemented and cross-connected, including object serialization, secure credential handling with

PBKDF2 cryptography, synchronous API stream processing, and multi-screen state orchestration. The project successfully demonstrates a secure and functional JavaFX application structure, bridging the gap between local data safety and external web service integration.

The architectural implementations carried out during the project lifecycle greatly improved the core understanding of:

- **Advanced JavaFX Layout Management:** Utilizing dynamic containers, custom component sizing, and modular scene navigation via decoupled controller classes.
- **Synchronous Web Communications:** Managing precise HTTP GET requests over external RESTful endpoints using native connection instances.
- **Complex Data Parsing:** Filtering and slicing nested JSON response arrays based on precise chronological intervals.
- **Secure Binary Data Persistence:** Moving from standard text streaming to automated object marshaling using ObjectOutputStream and ObjectInputStream frameworks to enforce local data integrity.

A major milestone of the application is the successfully integrated 5-day weather forecasting dashboard rendered dynamically below the real-time weather status. This feature highlights the application's ability to efficiently handle array iterations, process sub-string date representations, and seamlessly present multi-day prognostic data inside a responsive, horizontally-aligned user interface layout.

While the system successfully complies with standards of identity verification and local binary mapping, future iterations may incorporate:

- Comprehensive relational database management systems (RDBMS) integration such as SQLite or PostgreSQL for structured query processing.
- Automated multi-language localized resource bundles (i18n support).

14. REFERENCES

- **OpenWeatherMap API Documentation:** https://openweathermap.org/api
- **JavaFX Standard Documentation:** https://openjfx.io/
- **Oracle Java Core Documentation:** https://docs.oracle.com/en/java/
- **JSON Java Library Repository (org.json):** https://mvnrepository.com/artifact/org.json/json